

IFT436 - Algorithmes et structures de données

Devoir #1

Question 1 Une question d'échauffement

On pourrait résoudre le problème en triant d'abord le tableau à l'aide du tri-fusion, puis en retournant la k -ième plus petite valeur ainsi que son indice original. Il faudrait donc parcourir le tableau original une première fois pour créer un nouveau tableau A contenant des paires $(T[i], i)$, où la première composante correspond à la valeur et la seconde à son indice original dans T . On applique ensuite le tri-fusion en utilisant la première composante de chaque paire.

Comme le tri-fusion s'exécute en temps $O(n \log n)$ et que parcourir le tableau initial s'exécute en $O(n)$, cette approche permet de résoudre le problème en temps $O(n + n \log n) \approx O(n \log n)$.

Il serait peut-être possible de faire mieux en utilisant une méthode inspirée de Quicksort. Au lieu de trier complètement le tableau, on effectue seulement les partitions nécessaires pour trouver la k -ième plus petite valeur. Dependamment des pivots, au pire on aurait quelque chose comme $O(n^2)$, mais au mieux $O(n)$.

Question 2 Le problème de la sous-somme

- (a) Non, cet algorithme ne résout pas correctement le problème de la sous-somme.

Considérons le contre-exemple

$$S = \{1, 2, 3, 4, 5\} \quad \text{et} \quad t = 10.$$

Il existe une solution au problème, par exemple

$$S' = \{1, 4, 5\},$$

puisque

$$1 + 4 + 5 = 10.$$

Cependant, l'algorithme choisit toujours le plus grand élément restant dans S . Il choisit donc successivement 5, 4, puis 3. La valeur de q évolue ainsi :

$$q = 5, \quad q = 5 + 4 = 9, \quad q = 5 + 4 + 3 = 12.$$

Puisque $q \geq t$, la boucle se termine. Comme

$$q = 12 \neq 10,$$

l'algorithme retourne `null`, alors qu'une solution existe.

l'algorithme ne résout donc pas correctement le problème de la sous-somme.

- (b) (1) Oui, cet algorithme résout correctement le problème de la sous-somme.

L'algorithme examine tous les sous-ensembles $S' \subseteq S$. Si un sous-ensemble dont la somme vaut t existe, il sera nécessairement examiné par l'algorithme, qui le retournera. Si aucun sous-ensemble possède une somme égale à t , alors l'algorithme aura examiné tous les sous-ensembles sans trouver de solution et retournera `null`. L'algorithme est donc correct.

(2) Dans le pire cas, l'algorithme entre dans la boucle une fois pour chacun des sous-ensembles de S .

Pour chacun des n éléments de S , il existe deux possibilités lors de la construction d'un sous-ensemble : l'élément appartient au sous-ensemble ou il n'y appartient pas.

$$\underbrace{2 \times 2 \times 2 \times \dots \times 2}_{n \times \text{fois}} = 2^n$$

Le nombre total de sous-ensembles est donc

$$2^n.$$

De manière équivalente,

$$\sum_{k=0}^n \binom{n}{k} = 2^n.$$

Ainsi, dans le pire cas, l'algorithme entre dans la boucle **pour chaque** 2^n fois.

- (c) (1) L'algorithme énumère seulement les sous-ensembles de S contenant exactement k éléments.

Le nombre de façons de choisir k éléments parmi les $n = |S|$ éléments de S est donné par le coefficient binomial

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}.$$

Dans le pire cas, l'algorithme entre dans la boucle

$$\boxed{\binom{n}{k}}$$

fois.

(2) En général cette valeur est plus petite que 2^n , pour $n > 0$ pour $k \in \{1, 2, \dots, |S| - 1\}$.

On sait que

$$2^n = \sum_{i=0}^n \binom{n}{i}.$$

Le terme $\binom{n}{k}$ ne compte que les sous-ensembles de taille exactement k , tandis que 2^n compte tous les sous-ensembles possibles de S .

Par conséquent,

$$\boxed{\binom{n}{k} < 2^n}$$

pour $n > 0$ et $k < n$.

Or si $k = n$ alors $\binom{n}{k} = 2^n$ dans ce cas, la valeur est la même. Puisqu'on considère le pire cas, les deux algorithmes respectent $O(2^n)$.

Détail de notation : Je note $\binom{n}{k}$, où k parmi n est noté avec n en haut et k en bas. Je mentionne puisque je sais que certains font l'inverse.

Question 3 Puissances en temps logarithmique

- (a) On veut démontrer que, pour tout entier $k \geq 0$, l'algorithme `pow(x,k)` retourne bien x^k .
On procède par induction sur k .

Cas de base.

Si $k = 0$, l'algorithme retourne 1. Et, $x^0 = 1$.

Si $k = 1$, l'algorithme retourne x . Et, $x^1 = x$.

Donc l'algorithme est correct pour $k = 0$ et $k = 1$.

Hypothèse d'induction.

Supposons maintenant que, pour tout entier $j < k$, l'appel

`pow(x, j)`

retourne correctement x^j .

Considérons un entier $k \geq 2$. L'algorithme calcule

$$\text{tmp} = \text{pow}\left(x, \left\lfloor \frac{k}{2} \right\rfloor\right).$$

Comme

$$\left\lfloor \frac{k}{2} \right\rfloor < k,$$

l'hypothèse d'induction implique que

$$\text{tmp} = x^{\lfloor k/2 \rfloor}.$$

Il reste alors à considérer deux cas.

Cas 1 : k est pair.

Il existe un entier m tel que

$$k = 2m.$$

Dans ce cas,

$$\left\lfloor \frac{k}{2} \right\rfloor = m,$$

donc

$$\text{tmp} = x^m.$$

L'algorithme retourne

$$\text{tmp} \cdot \text{tmp} = x^m x^m = x^{2m} = x^k.$$

Cas 2 : k est impair.

Il existe un entier m tel que

$$k = 2m + 1.$$

Dans ce cas,

$$\left\lfloor \frac{k}{2} \right\rfloor = m,$$

donc

$$\text{tmp} = x^m.$$

L'algorithme retourne alors

$$\text{tmp} \cdot \text{tmp} \cdot x = x^m x^m x = x^{2m+1} = x^k.$$

Dans les deux cas, l'algorithme retourne correctement x^k .

Par induction, l'algorithme $\text{pow}(\mathbf{x}, \mathbf{k})$ calcule correctement x^k pour tout $k \geq 0$.

(b) Soit $r(k)$ le nombre total d'appels à la fonction pow , en comptant l'appel initial.

Lorsque $k = 1$, la fonction retourne directement x , donc

$$r(1) = 1.$$

Lorsque $k > 1$, la fonction effectue exactement un appel récursif avec l'argument $\lfloor k/2 \rfloor$. Puisque nous supposons que k est une puissance de 2, k est pair et nous avons

$$\left\lfloor \frac{k}{2} \right\rfloor = \frac{k}{2}, \quad \text{et} \quad r(k) = 1 + r\left(\frac{k}{2}\right).$$

Nous voulons démontrer que, pour toute puissance de 2,

$$r(k) = \log_2(k) + 1.$$

avec $k = 2^m$, et $m \geq 0$.

Nous procédons par induction sur m .

Cas de base : $m = 0$.

On a

$$k = 2^0 = 1.$$

Alors

$$r(1) = 1.$$

Et

$$\log_2(1) + 1 = 0 + 1 = 1.$$

La propriété est donc vraie pour $m = 0$.

Hypothèse d'induction.

Supposons que, pour un certain $m \geq 0$,

$$r(2^m) = m + 1.$$

Étape d'induction.

Considérons la puissance de 2 suivante, soit

$$k = 2^{m+1}.$$

Pour $k > 1$, l'appel courant à `pow(x,k)` compte pour un appel et la fonction effectue ensuite un appel récursif avec $k/2$. Par définition de r , on a donc

$$r(k) = 1 + r\left(\frac{k}{2}\right).$$

En remplaçant k par 2^{m+1} , on obtient

$$r(2^{m+1}) = 1 + r\left(\frac{2^{m+1}}{2}\right).$$

$$r(2^{m+1}) = 1 + r(2^m).$$

En utilisant l'hypothèse d'induction $r(2^m) = m + 1$,

$$r(2^{m+1}) = 1 + (m + 1) = m + 2.$$

Finalement,

$$m + 2 = \log_2(2^{m+1}) + 1.$$

On peut remplacer,

$$r(2^{m+1}) = \log_2(2^{m+1}) + 1.$$

Par induction, pour toute puissance de 2,

$$\boxed{r(k) = \log_2(k) + 1}.$$